

MPU



main.c

```

/* USER CODE BEGIN Header */
/**
 * *****
 * @file      : main.c
 * @brief     : Main program body
 * *****
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 * *****
 */
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"
#include "mpu.h"
#include <stdio.h>
#include <string.h>
/* Private includes -----*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/

```

```

CAN_HandleTypeDef hcan1;

I2C_HandleTypeDef hi2c1;

UART_HandleTypeDef huart2;

/* USER CODE BEGIN PV */
CAN_TxHeaderTypeDef TxHeader;
uint32_t TxMailbox;

uint8_t TxData[8];

char msg[150];
uint8_t FaultFlag = 0;
/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_CAN1_Init(void);
static void MX_I2C1_Init(void);
/* USER CODE BEGIN PFP */
void CAN_Send_MPU6050(void);
/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{

/* USER CODE BEGIN 1 */

/* USER CODE END 1 */

/* MCU Configuration-----*/

```

```
/* Reset of all peripherals, Initializes the Flash interface and the Systick. */
HAL_Init();
```

```
/* USER CODE BEGIN Init */
```

```
/* USER CODE END Init */
```

```
/* Configure the system clock */
SystemClock_Config();
```

```
/* USER CODE BEGIN SysInit */
```

```
/* USER CODE END SysInit */
```

```
/* Initialize all configured peripherals */
```

```
MX_GPIO_Init();
```

```
MX_USART2_UART_Init();
```

```
MX_CAN1_Init();
```

```
MX_I2C1_Init();
```

```
/* USER CODE BEGIN 2 */
```

```
MPU6050_Init();
```

```
if(HAL_CAN_Start(&hcan1) != HAL_OK)
{
    Error_Handler();
}
```

```
sprintf(msg,
        "MPU6050 CAN TX Started\r\n");
```

```
HAL_UART_Transmit(&huart2,
                  (uint8_t *)msg,
                  strlen(msg),
                  HAL_MAX_DELAY);
```

```
/* USER CODE END 2 */
```

```
/* Infinite loop */
```

```
/* USER CODE BEGIN WHILE */
```

```
while(1)
```

```
{
    CAN_Send_MPU6050();
```

```
    HAL_Delay(500);
```

```
}
```

```

}
/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage
    */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);

    /** Initializes the RCC Oscillators according to the specified parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    RCC_OscInitStruct.PLL.PLLM = 16;
    RCC_OscInitStruct.PLL.PLLN = 336;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV4;
    RCC_OscInitStruct.PLL.PLLQ = 2;
    RCC_OscInitStruct.PLL.PLLR = 2;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
    */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                                  |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
    {

```

```

    Error_Handler();
}
}

/**
 * @brief CAN1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_CAN1_Init(void)
{

    /* USER CODE BEGIN CAN1_Init 0 */

    /* USER CODE END CAN1_Init 0 */

    /* USER CODE BEGIN CAN1_Init 1 */

    /* USER CODE END CAN1_Init 1 */
    hcan1.Instance = CAN1;
    hcan1.Init.Prescaler = 6;
    hcan1.Init.Mode = CAN_MODE_NORMAL;
    hcan1.Init.SyncJumpWidth = CAN_SJW_1TQ;
    hcan1.Init.TimeSeg1 = CAN_BS1_11TQ;
    hcan1.Init.TimeSeg2 = CAN_BS2_2TQ;
    hcan1.Init.TimeTriggeredMode = DISABLE;
    hcan1.Init.AutoBusOff = ENABLE;
    hcan1.Init.AutoWakeUp = DISABLE;
    hcan1.Init.AutoRetransmission = DISABLE;
    hcan1.Init.ReceiveFifoLocked = DISABLE;
    hcan1.Init.TransmitFifoPriority = DISABLE;
    if (HAL_CAN_Init(&hcan1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN CAN1_Init 2 */

    /* USER CODE END CAN1_Init 2 */

}

/**
 * @brief I2C1 Initialization Function
 * @param None

```



```

* @retval None
*/
static void MX_I2C1_Init(void)
{

    /* USER CODE BEGIN I2C1_Init 0 */

    /* USER CODE END I2C1_Init 0 */

    /* USER CODE BEGIN I2C1_Init 1 */

    /* USER CODE END I2C1_Init 1 */
    hi2c1.Instance = I2C1;
    hi2c1.Init.ClockSpeed = 100000;
    hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
    hi2c1.Init.OwnAddress1 = 0;
    hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
    hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
    hi2c1.Init.OwnAddress2 = 0;
    hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
    hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
    if (HAL_I2C_Init(&hi2c1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN I2C1_Init 2 */

    /* USER CODE END I2C1_Init 2 */

}

/**
* @brief USART2 Initialization Function
* @param None
* @retval None
*/
static void MX_USART2_UART_Init(void)
{

    /* USER CODE BEGIN USART2_Init 0 */

    /* USER CODE END USART2_Init 0 */

    /* USER CODE BEGIN USART2_Init 1 */

```

```

/* USER CODE END USART2_Init 1 */
huart2.Instance = USART2;
huart2.Init.BaudRate = 115200;
huart2.Init.WordLength = UART_WORDLENGTH_8B;
huart2.Init.StopBits = UART_STOPBITS_1;
huart2.Init.Parity = UART_PARITY_NONE;
huart2.Init.Mode = UART_MODE_TX_RX;
huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
huart2.Init.OverSampling = UART_OVERSAMPLING_16;
if (HAL_UART_Init(&huart2) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN USART2_Init 2 */

/* USER CODE END USART2_Init 2 */

}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */

    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);

    /*Configure GPIO pin : B1_Pin */
    GPIO_InitStruct.Pin = B1_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;

```

```
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);
```

```
/*Configure GPIO pin : LD2_Pin */
GPIO_InitStruct.Pin = LD2_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);
```

```
/* USER CODE BEGIN MX_GPIO_Init_2 */
```

```
/* USER CODE END MX_GPIO_Init_2 */
}
```

```
/* USER CODE BEGIN 4 */
```

```
void CAN_Send_MPU6050(void)
{
    /* — Bus-Off recovery — */
    if (HAL_CAN_GetError(&hcan1) & HAL_CAN_ERROR_BOF)
    {
        HAL_CAN_Stop(&hcan1);
        HAL_Delay(100);
        if (HAL_CAN_Start(&hcan1) != HAL_OK)
        {
            sprintf(msg, "CAN Restart Failed\r\n");
            HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
            return;
        }
        sprintf(msg, "CAN Bus-Off Recovered\r\n");
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
    }
}
```

```
MPU6050_Read_All();
```

```
int16_t Ax_tx = (int16_t)(Ax * 100);
int16_t Ay_tx = (int16_t)(Ay * 100);
int16_t Az_tx = (int16_t)(Az * 100);
int16_t Gx_tx = (int16_t)(Gx * 100);
int16_t Gy_tx = (int16_t)(Gy * 100);
int16_t Gz_tx = (int16_t)(Gz * 100);
```

```
TxHeader.ExtId      = 0;
TxHeader.IDE        = CAN_ID_STD;
```

```

TxHeader.RTR          = CAN_RTR_DATA;
TxHeader.DLC          = 6;
TxHeader.TransmitGlobalTime = DISABLE;

/* — Accelerometer — */
TxHeader.StdId = 0x100;
TxData[0] = (uint8_t)(Ax_tx >> 8);
TxData[1] = (uint8_t)(Ax_tx);
TxData[2] = (uint8_t)(Ay_tx >> 8);
TxData[3] = (uint8_t)(Ay_tx);
TxData[4] = (uint8_t)(Az_tx >> 8);
TxData[5] = (uint8_t)(Az_tx);

uint32_t tick = HAL_GetTick();
while (HAL_CAN_GetTxMailboxesFreeLevel(&hcan1) == 0)
    if (HAL_GetTick() - tick > 10) break;

if (HAL_CAN_AddTxMessage(&hcan1, &TxHeader, TxData, &TxMailbox) != HAL_OK)
{
    sprintf(msg, "ACC TX Error=0x%08lX\r\n", HAL_CAN_GetError(&hcan1));
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
    HAL_CAN_ResetError(&hcan1); /* clear error flags */
}
else
{
    sprintf(msg, "ACC Sent: AX=%.2f AY=%.2f AZ=%.2f\r\n", Ax, Ay, Az);
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
}

HAL_Delay(10);

/* — Gyroscope — */
TxHeader.StdId = 0x101;
TxData[0] = (uint8_t)(Gx_tx >> 8);
TxData[1] = (uint8_t)(Gx_tx);
TxData[2] = (uint8_t)(Gy_tx >> 8);
TxData[3] = (uint8_t)(Gy_tx);
TxData[4] = (uint8_t)(Gz_tx >> 8);
TxData[5] = (uint8_t)(Gz_tx);

tick = HAL_GetTick();
while (HAL_CAN_GetTxMailboxesFreeLevel(&hcan1) == 0)
    if (HAL_GetTick() - tick > 10) break;

```

```

if (HAL_CAN_AddTxMessage(&hcan1, &TxHeader, TxData, &TxMailbox) != HAL_OK)
{
    sprintf(msg, "GYR TX Error=0x%08lX\r\n", HAL_CAN_GetError(&hcan1));
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
    HAL_CAN_ResetError(&hcan1);
}
else
{
    sprintf(msg, "GYR Sent: GX=%.2f GY=%.2f GZ=%.2f\r\n", Gx, Gy, Gz);
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
}
}

```

```

/* USER CODE END 4 */

```

```

/**

```

```

 * @brief This function is executed in case of error occurrence.
 * @retval None
 */

```

```

void Error_Handler(void)

```

```

{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

```

```

#ifdef USE_FULL_ASSERT

```

```

/**

```

```

 * @brief Reports the name of the source file and the source line number
 *         where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */

```

```

void assert_failed(uint8_t *file, uint32_t line)

```

```

{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
       ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}

```

```
#endif /* USE_FULL_ASSERT */
```

mpu6050.h

## MPU 6050 - MPU.H

```
#ifndef MPU_H
#define MPU_H
#include "main.h"
#define MPU6050_ADDR (0x68 << 1)
extern float Ax, Ay, Az;
extern float Gx, Gy, Gz;
void MPU6050_Init(void);
void MPU6050_Read_All(void);
#endif
```



mpu6050.c

## MPU 6050 - mpu.c

```
#include "mpu.h"
extern I2C_HandleTypeDef hi2c1;
#define MPU6050_ADDR (0x68 << 1)
float Ax, Ay, Az;
float Gx, Gy, Gz;
void MPU6050_Init(void)
{
    uint8_t data;
    data = 0x00;
    HAL_I2C_Mem_Write(&hi2c1,
        MPU6050_ADDR,
        0x6B,
        1,
        &data,
        1,
        HAL_MAX_DELAY);
    data = 0x00;
    HAL_I2C_Mem_Write(&hi2c1,
        MPU6050_ADDR,
        0x1C,
        1,
        &data,
        1,
        HAL_MAX_DELAY);
    HAL_I2C_Mem_Write(&hi2c1,
        MPU6050_ADDR,
        0x1B,
        1,
        &data,
        1,
        HAL_MAX_DELAY);
}
void MPU6050_Read_All(void)
{
    uint8_t Rec_Data[14];
    int16_t Accel_X_RAW;
    int16_t Accel_Y_RAW;
    int16_t Accel_Z_RAW;
    int16_t Gyro_X_RAW;
    int16_t Gyro_Y_RAW;
    int16_t Gyro_Z_RAW;
    HAL_I2C_Mem_Read(&hi2c1,
        MPU6050_ADDR,
        0x3B,
        1,
        Rec_Data,
        14,
```

```
        HAL_MAX_DELAY);
Accel_X_RAW =
    (Rec_Data[0] << 8 | Rec_Data[1]);
Accel_Y_RAW =
    (Rec_Data[2] << 8 | Rec_Data[3]);
Accel_Z_RAW =
    (Rec_Data[4] << 8 | Rec_Data[5]);
Gyro_X_RAW =
    (Rec_Data[8] << 8 | Rec_Data[9]);
Gyro_Y_RAW =
    (Rec_Data[10] << 8 | Rec_Data[11]);
Gyro_Z_RAW =
    (Rec_Data[12] << 8 | Rec_Data[13]);
Ax = (float)Accel_X_RAW / 16384.0f;
Ay = (float)Accel_Y_RAW / 16384.0f;
Az = (float)Accel_Z_RAW / 16384.0f;
Gx = (float)Gyro_X_RAW / 131.0f;
Gy = (float)Gyro_Y_RAW / 131.0f;
Gz = (float)Gyro_Z_RAW / 131.0f;
}
```

DHT11



dht\_main.c

```

/* USER CODE BEGIN Header */
/**
 * *****
 * @file      : main.c
 * @brief     : Main program body
 * *****
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 * *****
 */
/* USER CODE END Header */
/* Includes ----- */
#include "main.h"
#include "dht11.h"
#include <stdio.h>
#include <string.h>
/* Private includes ----- */
/* USER CODE BEGIN Includes */
/* USER CODE END Includes */
/* Private typedef ----- */
/* USER CODE BEGIN PTD */
/* USER CODE END PTD */
/* Private define ----- */
/* USER CODE BEGIN PD */
/* USER CODE END PD */
/* Private macro ----- */
/* USER CODE BEGIN PM */
/* USER CODE END PM */
/* Private variables ----- */
CAN_HandleTypeDef hcan1;
TIM_HandleTypeDef htim2;
UART_HandleTypeDef huart2;
/* USER CODE BEGIN PV */
CAN_TxHeaderTypeDef TxHeader;
uint32_t TxMailbox;
uint8_t TxData[8];
uint8_t FaultFlag = 0;
char uart_msg[100];
/* USER CODE END PV */
/* Private function prototypes ----- */
void SystemClock_Config(void);
static void MX_GPIO_Init(void);

```

```

static void MX_USART2_UART_Init(void);
static void MX_CAN1_Init(void);
static void MX_TIM2_Init(void);
/* USER CODE BEGIN PFP */
/* USER CODE END PFP */
/* Private user code ----- */
/* USER CODE BEGIN 0 */
void CAN_DHT11_Transmit(void);
/* USER CODE END 0 */
/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */
    /* USER CODE END 1 */
    /* MCU Configuration----- */
    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();
    /* USER CODE BEGIN Init */
    /* USER CODE END Init */
    /* Configure the system clock */
    SystemClock_Config();
    /* USER CODE BEGIN SysInit */
    /* USER CODE END SysInit */
    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    MX_CAN1_Init();
    MX_TIM2_Init();
    /* USER CODE BEGIN 2 */
    HAL_TIM_Base_Start(&htim2);
    if(HAL_CAN_Start(&hcan1) != HAL_OK)
    {
        Error_Handler();
    }
    sprintf(uart_msg, "CAN Started\r\n");
    HAL_UART_Transmit(&huart2,
        (uint8_t*)uart_msg,
        strlen(uart_msg),
        HAL_MAX_DELAY);
    /* USER CODE END 2 */
    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
        /* USER CODE END WHILE */
        CAN_DHT11_Transmit();

```



```

        HAL_Delay(500);
    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}
/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    /** Configure the main internal regulator output voltage
    */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);
    /** Initializes the RCC Oscillators according to the specified parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    RCC_OscInitStruct.PLL.PLLM = 16;
    RCC_OscInitStruct.PLL.PLLN = 336;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV4;
    RCC_OscInitStruct.PLL.PLLQ = 2;
    RCC_OscInitStruct.PLL.PLLR = 2;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
    /** Initializes the CPU, AHB and APB buses clocks
    */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
        |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
    {
        Error_Handler();
    }
}
/**
 * @brief CAN1 Initialization Function

```

```

* @param None
* @retval None
*/
static void MX_CAN1_Init(void)
{
    /* USER CODE BEGIN CAN1_Init 0 */
    /* USER CODE END CAN1_Init 0 */
    /* USER CODE BEGIN CAN1_Init 1 */
    /* USER CODE END CAN1_Init 1 */
    hcan1.Instance = CAN1;
    hcan1.Init.Prescaler = 6;
    hcan1.Init.Mode = CAN_MODE_NORMAL;
    hcan1.Init.SyncJumpWidth = CAN_SJW_1TQ;
    hcan1.Init.TimeSeg1 = CAN_BS1_11TQ;
    hcan1.Init.TimeSeg2 = CAN_BS2_2TQ;
    hcan1.Init.TimeTriggeredMode = DISABLE;
    hcan1.Init.AutoBusOff = DISABLE;
    hcan1.Init.AutoWakeUp = DISABLE;
    hcan1.Init.AutoRetransmission = DISABLE;
    hcan1.Init.ReceiveFifoLocked = DISABLE;
    hcan1.Init.TransmitFifoPriority = DISABLE;
    if (HAL_CAN_Init(&hcan1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN CAN1_Init 2 */
    /* USER CODE END CAN1_Init 2 */
}
/**
* @brief TIM2 Initialization Function
* @param None
* @retval None
*/
static void MX_TIM2_Init(void)
{
    /* USER CODE BEGIN TIM2_Init 0 */
    /* USER CODE END TIM2_Init 0 */
    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};
    /* USER CODE BEGIN TIM2_Init 1 */
    /* USER CODE END TIM2_Init 1 */
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 84-1;
    htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim2.Init.Period = 4294967295;
    htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
    {

```

```

    Error_Handler();
}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
{
    Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM2_Init 2 */
/* USER CODE END TIM2_Init 2 */
}
/**
 * @brief USART2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_USART2_UART_Init(void)
{
    /* USER CODE BEGIN USART2_Init 0 */
    /* USER CODE END USART2_Init 0 */
    /* USER CODE BEGIN USART2_Init 1 */
    /* USER CODE END USART2_Init 1 */
    huart2.Instance = USART2;
    huart2.Init.BaudRate = 115200;
    huart2.Init.WordLength = UART_WORDLENGTH_8B;
    huart2.Init.StopBits = UART_STOPBITS_1;
    huart2.Init.Parity = UART_PARITY_NONE;
    huart2.Init.Mode = UART_MODE_TX_RX;
    huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart2.Init.OverSampling = UART_OVERSAMPLING_16;
    if (HAL_UART_Init(&huart2) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN USART2_Init 2 */
    /* USER CODE END USART2_Init 2 */
}
/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{

```

```

GPIO_InitTypeDef GPIO_InitStruct = {0};
/* USER CODE BEGIN MX_GPIO_Init_1 */
/* USER CODE END MX_GPIO_Init_1 */
/* GPIO Ports Clock Enable */
__HAL_RCC_GPIOC_CLK_ENABLE();
__HAL_RCC_GPIOH_CLK_ENABLE();
__HAL_RCC_GPIOA_CLK_ENABLE();
__HAL_RCC_GPIOB_CLK_ENABLE();
/*Configure GPIO pin Output Level */
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1|LD2_Pin, GPIO_PIN_RESET);
/*Configure GPIO pin : B1_Pin */
GPIO_InitStruct.Pin = B1_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);
/*Configure GPIO pins : PA1 LD2_Pin */
GPIO_InitStruct.Pin = GPIO_PIN_1|LD2_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
/* USER CODE BEGIN MX_GPIO_Init_2 */
/* USER CODE END MX_GPIO_Init_2 */
}
/* USER CODE BEGIN 4 */
void CAN_DHT11_Transmit(void)
{
    FaultFlag = 0;
    DHT11_Read_Data();
    if((DHT_Temperature == 0) &&
        (DHT_Humidity == 0))
    {
        FaultFlag = 0x01;
    }
    TxHeader.StdId = 0x201;
    TxHeader.ExtId = 0;
    TxHeader.IDE = CAN_ID_STD;
    TxHeader.RTR = CAN_RTR_DATA;
    TxHeader.DLC = 2;
    TxHeader.TransmitGlobalTime = DISABLE;
    TxData[0] = (uint8_t)DHT_Temperature;
    TxData[1] = (uint8_t)DHT_Humidity;
    if (HAL_CAN_AddTxMessage(&hcan1,
        &TxHeader,
        TxData,
        &TxMailbox) == HAL_OK)
    {
        sprintf(uart_msg,
            "TX: Temp=%d C Hum=%d %%\r\n",

```

```

        TxData[0],
        TxData[1]);
    HAL_UART_Transmit(&huart2,
        (uint8_t*)uart_msg,
        strlen(uart_msg),
        HAL_MAX_DELAY);
}
else
{
    sprintf(uart_msg,
        "CAN TX Failed\r\n");
    HAL_UART_Transmit(&huart2,
        (uint8_t*)uart_msg,
        strlen(uart_msg),
        HAL_MAX_DELAY);
}
}
/* USER CODE END 4 */
/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}

#endif /* USE_FULL_ASSERT */

```



dht11.h

```
#ifndef DHT11_H
#define DHT11_H
#include "main.h"
void DHT11_Start(void);
uint8_t DHT11_Check_Response(void);
uint8_t DHT11_Read(void);
void DHT11_Read_Data(void);
extern float DHT_Temperature;
extern float DHT_Humidity;
#endif
```



dht11.c

```

#include "dht11.h"
extern TIM_HandleTypeDef htim2;
#define DHT11_PORT GPIOA
#define DHT11_PIN GPIO_PIN_1
float DHT_Temperature = 0;
float DHT_Humidity = 0;
/* GPIO Configuration */
static void Set_Pin_Output(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOx, &GPIO_InitStruct);
}
static void Set_Pin_Input(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOx, &GPIO_InitStruct);
}
/* Microsecond Delay */
void delay_us(uint16_t us)
{
    __HAL_TIM_SET_COUNTER(&htim2, 0);
    while (__HAL_TIM_GET_COUNTER(&htim2) < us);
}
void DHT11_Start(void)
{
    Set_Pin_Output(DHT11_PORT, DHT11_PIN);
    HAL_GPIO_WritePin(DHT11_PORT,
        DHT11_PIN,
        GPIO_PIN_RESET);
    HAL_Delay(18);
    HAL_GPIO_WritePin(DHT11_PORT,
        DHT11_PIN,
        GPIO_PIN_SET);
    delay_us(30);
    Set_Pin_Input(DHT11_PORT, DHT11_PIN);
}
uint8_t DHT11_Check_Response(void)
{
    uint8_t Response = 0;
    uint32_t timeout = 10000;
    delay_us(40);
    if (!(HAL_GPIO_ReadPin(DHT11_PORT, DHT11_PIN)))

```

```

{
    delay_us(80);
    if (HAL_GPIO_ReadPin(DHT11_PORT, DHT11_PIN))
    {
        Response = 1;
    }
}
timeout = 10000;
while (HAL_GPIO_ReadPin(DHT11_PORT, DHT11_PIN))
{
    if (timeout-- == 0)
    {
        return 0;
    }
}
return Response;
}
uint8_t DHT11_Read(void)
{
    uint8_t i = 0;
    uint8_t j;
    uint32_t timeout;
    for (j = 0; j < 8; j++)
    {
        timeout = 10000;
        while (!(HAL_GPIO_ReadPin(DHT11_PORT, DHT11_PIN)))
        {
            if (timeout-- == 0)
            {
                return 0;
            }
        }
        delay_us(40);
        if (!(HAL_GPIO_ReadPin(DHT11_PORT, DHT11_PIN)))
        {
            i &= ~(1 << (7 - j));
        }
        else
        {
            i |= (1 << (7 - j));
            timeout = 10000;
            while (HAL_GPIO_ReadPin(DHT11_PORT, DHT11_PIN))
            {
                if (timeout-- == 0)
                {
                    return 0;
                }
            }
        }
    }
}

```

```

    }
    return i;
}
void DHT11_Read_Data(void)
{
    DHT_Temperature = 0;
    DHT_Humidity = 0;
    uint8_t Rh_byte1;
    uint8_t Rh_byte2;
    uint8_t Temp_byte1;
    uint8_t Temp_byte2;
    uint8_t SUM;
    DHT11_Start();
    if (DHT11_Check_Response())
    {
        Rh_byte1 = DHT11_Read();
        Rh_byte2 = DHT11_Read();
        Temp_byte1 = DHT11_Read();
        Temp_byte2 = DHT11_Read();
        SUM = DHT11_Read();
        if (SUM == (Rh_byte1 +
                    Rh_byte2 +
                    Temp_byte1 +
                    Temp_byte2))
        {
            DHT_Humidity = (float)Rh_byte1;
            DHT_Temperature = (float)Temp_byte1;
        }
    }
}

```

DS18B20



ds18b20\_main.c

```

/* USER CODE BEGIN Header */
/**
 * *****
 * @file      : main.c
 * @brief     : Main program body
 * *****
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 * *****
 */
/* USER CODE END Header */
/* Includes ----- */
#include "main.h"
#include <stdio.h>
#include <string.h>
#include "ds18b20.h"
/* Private includes ----- */
/* USER CODE BEGIN Includes */
/* USER CODE END Includes */
/* Private typedef ----- */
/* USER CODE BEGIN PTD */
/* USER CODE END PTD */
/* Private define ----- */
/* USER CODE BEGIN PD */
/* USER CODE END PD */
/* Private macro ----- */
/* USER CODE BEGIN PM */
/* USER CODE END PM */
/* Private variables ----- */
CAN_HandleTypeDef hcan1;
TIM_HandleTypeDef htim2;
UART_HandleTypeDef huart2;
/* USER CODE BEGIN PV */
CAN_TxHeaderTypeDef TxHeader;
uint32_t TxMailbox;
uint8_t TxData[8];
char msg[100];
uint8_t FaultFlag = 0;
/* USER CODE END PV */
/* Private function prototypes ----- */
void SystemClock_Config(void);
static void MX_GPIO_Init(void);

```



```

static void MX_USART2_UART_Init(void);
static void MX_CAN1_Init(void);
static void MX_TIM2_Init(void);
/* USER CODE BEGIN PFP */
/* USER CODE END PFP */
/* Private user code ----- */
/* USER CODE BEGIN 0 */
void CAN_DS18B20_Transmit(void);
/* USER CODE END 0 */
/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */
    /* USER CODE END 1 */
    /* MCU Configuration----- */
    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();
    /* USER CODE BEGIN Init */
    /* USER CODE END Init */
    /* Configure the system clock */
    SystemClock_Config();
    /* USER CODE BEGIN SysInit */
    /* USER CODE END SysInit */
    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    MX_CAN1_Init();
    MX_TIM2_Init();
    /* USER CODE BEGIN 2 */
    HAL_TIM_Base_Start(&htim2);
    if(HAL_CAN_Start(&hcan1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE END 2 */
    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
        /* USER CODE END WHILE */
        CAN_DS18B20_Transmit();
        HAL_Delay(500);
        /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}

```

```

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    /** Configure the main internal regulator output voltage
    */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);
    /** Initializes the RCC Oscillators according to the specified parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    RCC_OscInitStruct.PLL.PLLM = 16;
    RCC_OscInitStruct.PLL.PLLN = 336;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV4;
    RCC_OscInitStruct.PLL.PLLQ = 2;
    RCC_OscInitStruct.PLL.PLLR = 2;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
    /** Initializes the CPU, AHB and APB buses clocks
    */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYCLK
                                |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
    {
        Error_Handler();
    }
}
/**
 * @brief CAN1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_CAN1_Init(void)
{

```

```

/* USER CODE BEGIN CAN1_Init 0 */
/* USER CODE END CAN1_Init 0 */
/* USER CODE BEGIN CAN1_Init 1 */
/* USER CODE END CAN1_Init 1 */
hcan1.Instance = CAN1;
hcan1.Init.Prescaler = 6;
hcan1.Init.Mode = CAN_MODE_NORMAL;
hcan1.Init.SyncJumpWidth = CAN_SJW_1TQ;
hcan1.Init.TimeSeg1 = CAN_BS1_11TQ;
hcan1.Init.TimeSeg2 = CAN_BS2_2TQ;
hcan1.Init.TimeTriggeredMode = DISABLE;
hcan1.Init.AutoBusOff = DISABLE;
hcan1.Init.AutoWakeUp = DISABLE;
hcan1.Init.AutoRetransmission = DISABLE;
hcan1.Init.ReceiveFifoLocked = DISABLE;
hcan1.Init.TransmitFifoPriority = DISABLE;
if (HAL_CAN_Init(&hcan1) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN CAN1_Init 2 */
/* USER CODE END CAN1_Init 2 */
}
/**
 * @brief TIM2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM2_Init(void)
{
    /* USER CODE BEGIN TIM2_Init 0 */
    /* USER CODE END TIM2_Init 0 */
    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};
    /* USER CODE BEGIN TIM2_Init 1 */
    /* USER CODE END TIM2_Init 1 */
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 84-1;
    htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim2.Init.Period = 4294967295;
    htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
    {
        Error_Handler();
    }
    sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
    if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
    {

```

```

    Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM2_Init 2 */
/* USER CODE END TIM2_Init 2 */
}
/**
 * @brief USART2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_USART2_UART_Init(void)
{
    /* USER CODE BEGIN USART2_Init 0 */
    /* USER CODE END USART2_Init 0 */
    /* USER CODE BEGIN USART2_Init 1 */
    /* USER CODE END USART2_Init 1 */
    huart2.Instance = USART2;
    huart2.Init.BaudRate = 115200;
    huart2.Init.WordLength = UART_WORDLENGTH_8B;
    huart2.Init.StopBits = UART_STOPBITS_1;
    huart2.Init.Parity = UART_PARITY_NONE;
    huart2.Init.Mode = UART_MODE_TX_RX;
    huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart2.Init.OverSampling = UART_OVERSAMPLING_16;
    if (HAL_UART_Init(&huart2) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN USART2_Init 2 */
    /* USER CODE END USART2_Init 2 */
}
/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */
    /* USER CODE END MX_GPIO_Init_1 */
    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOC_CLK_ENABLE();

```

```

__HAL_RCC_GPIOH_CLK_ENABLE();
__HAL_RCC_GPIOA_CLK_ENABLE();
__HAL_RCC_GPIOB_CLK_ENABLE();
/*Configure GPIO pin Output Level */
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1|LD2_Pin, GPIO_PIN_RESET);
/*Configure GPIO pin : B1_Pin */
GPIO_InitStruct.Pin = B1_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);
/*Configure GPIO pins : PA1 LD2_Pin */
GPIO_InitStruct.Pin = GPIO_PIN_1|LD2_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
/* USER CODE BEGIN MX_GPIO_Init_2 */
/* USER CODE END MX_GPIO_Init_2 */
}
/* USER CODE BEGIN 4 */
void CAN_DS18B20_Transmit(void)
{
    float temp;
    temp = DS18B20_GetTemp();
    FaultFlag = 0;
    /* DS18B20 fault detection */
    if(temp == -127.0f || temp == 85.0f)
    {
        FaultFlag = 0x01;
    }
    int16_t temp_tx = (int16_t)(temp * 100);
    TxHeader.StdId = 0x301;
    TxHeader.ExtId = 0;
    TxHeader.IDE = CAN_ID_STD;
    TxHeader.RTR = CAN_RTR_DATA;
    TxHeader.DLC = 3;
    TxHeader.TransmitGlobalTime = DISABLE;
    TxData[0] = temp_tx >> 8;
    TxData[1] = temp_tx;
    TxData[2] = FaultFlag;
    HAL_CAN_AddTxMessage(&hcan1,
        &TxHeader,
        TxData,
        &TxMailbox);
    sprintf(msg,
        "DS18B20 = %.2f C Fault=%d\r\n",
        temp,
        FaultFlag);
    HAL_UART_Transmit(&huart2,

```

```

        (uint8_t*)msg,
        strlen(msg),
        HAL_MAX_DELAY);
    }
/* USER CODE END 4 */
/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
        /* USER CODE END Error_Handler_Debug */
    }
#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 *        where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
       ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```

ds18b20.h

```
#ifndef DS18B20_H
#define DS18B20_H
#include "main.h"
extern float DS18B20_Temp;
void delay_us(uint16_t us);
uint8_t DS18B20_Start(void);
void DS18B20_Write(uint8_t data);
uint8_t DS18B20_Read(void);
float DS18B20_GetTemp(void);
#endif
```



ds18b20.c

```

#include "ds18b20.h"
extern TIM_HandleTypeDef htim2;
#define DS18B20_PORT GPIOA
#define DS18B20_PIN GPIO_PIN_1
float DS18B20_Temp = 0.0f;
static uint8_t conversion_started = 0;
static uint32_t conversion_tick = 0;
/*-----*/
/* GPIO Functions */
/*-----*/

static void Set_Pin_Output(GPIO_TypeDef *GPIOx,
                           uint16_t GPIO_Pin)
{
    GPIO_InitTypeDef GPIO_InitStructure = {0};
    GPIO_InitStructure.Pin = GPIO_Pin;
    GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_OD;
    GPIO_InitStructure.Pull = GPIO_NOPULL;
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOx, &GPIO_InitStructure);
}

static void Set_Pin_Input(GPIO_TypeDef *GPIOx,
                          uint16_t GPIO_Pin)
{
    GPIO_InitTypeDef GPIO_InitStructure = {0};
    GPIO_InitStructure.Pin = GPIO_Pin;
    GPIO_InitStructure.Mode = GPIO_MODE_INPUT;
    GPIO_InitStructure.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOx, &GPIO_InitStructure);
}
/*-----*/
/* Microsecond Delay */
/*-----*/

void delay_us(uint16_t us)
{
    __HAL_TIM_SET_COUNTER(&htim2, 0);
    while (__HAL_TIM_GET_COUNTER(&htim2) < us);
}
/*-----*/
/* Reset + Presence Detection */
/*-----*/

uint8_t DS18B20_Start(void)
{
    uint8_t Response = 0;
    Set_Pin_Output(DS18B20_PORT,
                  DS18B20_PIN);
    HAL_GPIO_WritePin(DS18B20_PORT,
                     DS18B20_PIN,
                     GPIO_PIN_RESET);
    delay_us(480);
}

```

```

Set_Pin_Input(DS18B20_PORT,
              DS18B20_PIN);
delay_us(80);
if (!(HAL_GPIO_ReadPin(DS18B20_PORT,
                      DS18B20_PIN)))
{
    Response = 1;
}
delay_us(400);
return Response;
}
/*-----*/
/* Write Byte */
/*-----*/
void DS18B20_Write(uint8_t data)
{
    for(uint8_t i = 0; i < 8; i++)
    {
        Set_Pin_Output(DS18B20_PORT,
                       DS18B20_PIN);
        HAL_GPIO_WritePin(DS18B20_PORT,
                          DS18B20_PIN,
                          GPIO_PIN_RESET);
        if(data & (1 << i))
        {
            delay_us(2);
            Set_Pin_Input(DS18B20_PORT,
                          DS18B20_PIN);
            delay_us(60);
        }
        else
        {
            delay_us(60);
            Set_Pin_Input(DS18B20_PORT,
                          DS18B20_PIN);
            delay_us(2);
        }
    }
}
/*-----*/
/* Read Byte */
/*-----*/
uint8_t DS18B20_Read(void)
{
    uint8_t value = 0;
    for(uint8_t i = 0; i < 8; i++)
    {
        Set_Pin_Output(DS18B20_PORT,
                       DS18B20_PIN);

```

```

    HAL_GPIO_WritePin(DS18B20_PORT,
                      DS18B20_PIN,
                      GPIO_PIN_RESET);
    delay_us(2);
    Set_Pin_Input(DS18B20_PORT,
                  DS18B20_PIN);
    delay_us(8);
    if(HAL_GPIO_ReadPin(DS18B20_PORT,
                        DS18B20_PIN))
    {
        value |= (1 << i);
    }
    delay_us(60);
}
return value;
}
/*-----*/
/* Read Temperature */
/*-----*/
float DS18B20_GetTemp(void)
{
    uint8_t Temp_L;
    uint8_t Temp_H;
    int16_t TEMP;
    if(conversion_started == 0)
    {
        if(DS18B20_Start())
        {
            DS18B20_Write(0xCC);
            DS18B20_Write(0x44);
            conversion_tick = HAL_GetTick();
            conversion_started = 1;
        }
        return DS18B20_Temp;
    }
    if((HAL_GetTick() - conversion_tick) < 750)
    {
        return DS18B20_Temp;
    }
    if(!DS18B20_Start())
    {
        conversion_started = 0;
        return DS18B20_Temp;
    }
    DS18B20_Write(0xCC);
    DS18B20_Write(0xBE);
    Temp_L = DS18B20_Read();
    Temp_H = DS18B20_Read();
    TEMP = (Temp_H << 8) | Temp_L;

```

```
DS18B20_Temp = (float)TEMP / 16.0f;  
if(DS18B20_Start()  
{  
    DS18B20_Write(0xCC);  
    DS18B20_Write(0x44);  
    conversion_tick = HAL_GetTick();  
    conversion_started = 1;  
}  
else  
{  
    conversion_started = 0;  
}  
return DS18B20_Temp;  
}
```

SPEED



MAIN.C



```

/* USER CODE BEGIN Header */
/**
 * *****
 * @file      : main.c
 * @brief     : Main program body
 * *****
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 * *****
 */
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"
#include <stdio.h>
/* Private includes -----*/
/* USER CODE BEGIN Includes */
/* USER CODE END Includes */
/* Private typedef -----*/
/* USER CODE BEGIN PTD */
/* USER CODE END PTD */
/* Private define -----*/
/* USER CODE BEGIN PD */
/* USER CODE END PD */
/* Private macro -----*/
/* USER CODE BEGIN PM */
/* USER CODE END PM */
/* Private variables -----*/
CAN_HandleTypeDef hcan1;
TIM_HandleTypeDef htim2;
UART_HandleTypeDef huart2;
/* USER CODE BEGIN PV */
CAN_TxHeaderTypeDef TxHeader;
uint32_t TxMailbox;
uint8_t TxData[2];
uint16_t speed = 50;
/* USER CODE END PV */
/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_CAN1_Init(void);
static void MX_TIM2_Init(void);

```

```

/* USER CODE BEGIN PFP */
/* USER CODE END PFP */
/* Private user code -----*/
/* USER CODE BEGIN 0 */
int __io_putchar(int ch)
{
    HAL_UART_Transmit(&huart2, (uint8_t*)&ch, 1, HAL_MAX_DELAY);
    return ch;
}
/* USER CODE END 0 */
/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */
    /* USER CODE END 1 */
    /* MCU Configuration-----*/
    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();
    /* USER CODE BEGIN Init */
    /* USER CODE END Init */
    /* Configure the system clock */
    SystemClock_Config();
    /* USER CODE BEGIN SysInit */
    /* USER CODE END SysInit */
    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    MX_CAN1_Init();
    MX_TIM2_Init();
    if(HAL_CAN_Start(&hcan1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN 2 */
    TxHeader.StdId = 0x401;
    TxHeader.ExtId = 0;
    TxHeader.IDE = CAN_ID_STD;
    TxHeader.RTR = CAN_RTR_DATA;
    TxHeader.DLC = 2; // Only 2 bytes for speed
    TxHeader.TransmitGlobalTime = DISABLE;
    /* USER CODE END 2 */
    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
        speed++;
    }
}

```

```

/* Pack Speed into 2 bytes */
TxData[0] = (speed >> 8) & 0xFF;
TxData[1] = speed & 0xFF;
if(HAL_CAN_AddTxMessage(&hcan1,
    &TxHeader,
    TxData,
    &TxMailbox) != HAL_OK)
{
    printf("CAN TX Failed\r\n");
}
else
{
    printf("Speed = %d km/h\r\n",
        speed);
}
HAL_Delay(500);
}
}
/* USER CODE END 3 */
/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    /** Configure the main internal regulator output voltage
     */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);
    /** Initializes the RCC Oscillators according to the specified parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    RCC_OscInitStruct.PLL.PLLM = 16;
    RCC_OscInitStruct.PLL.PLLN = 336;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV4;
    RCC_OscInitStruct.PLL.PLLQ = 2;
    RCC_OscInitStruct.PLL.PLLR = 2;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
    /** Initializes the CPU, AHB and APB buses clocks

```

```

*/
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYCLK
                             |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
{
    Error_Handler();
}
}
/**
 * @brief CAN1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_CAN1_Init(void)
{
    /* USER CODE BEGIN CAN1_Init 0 */
    /* USER CODE END CAN1_Init 0 */
    /* USER CODE BEGIN CAN1_Init 1 */
    /* USER CODE END CAN1_Init 1 */
    hcan1.Instance = CAN1;
    hcan1.Init.Prescaler = 6;
    hcan1.Init.Mode = CAN_MODE_NORMAL;
    hcan1.Init.SyncJumpWidth = CAN_SJW_1TQ;
    hcan1.Init.TimeSeg1 = CAN_BS1_11TQ;
    hcan1.Init.TimeSeg2 = CAN_BS2_2TQ;
    hcan1.Init.TimeTriggeredMode = DISABLE;
    hcan1.Init.AutoBusOff = ENABLE;
    hcan1.Init.AutoWakeUp = DISABLE;
    hcan1.Init.AutoRetransmission = DISABLE;
    hcan1.Init.ReceiveFifoLocked = DISABLE;
    hcan1.Init.TransmitFifoPriority = DISABLE;
    if (HAL_CAN_Init(&hcan1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN CAN1_Init 2 */
    /* USER CODE END CAN1_Init 2 */
}
/**
 * @brief TIM2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM2_Init(void)
{

```

```

/* USER CODE BEGIN TIM2_Init 0 */
/* USER CODE END TIM2_Init 0 */
TIM_ClockConfigTypeDef sClockSourceConfig = {0};
TIM_MasterConfigTypeDef sMasterConfig = {0};
/* USER CODE BEGIN TIM2_Init 1 */
/* USER CODE END TIM2_Init 1 */
htim2.Instance = TIM2;
htim2.Init.Prescaler = 0;
htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
htim2.Init.Period = 4294967295;
htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
{
    Error_Handler();
}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
{
    Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM2_Init 2 */
/* USER CODE END TIM2_Init 2 */
}
/**
 * @brief USART2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_USART2_UART_Init(void)
{
    /* USER CODE BEGIN USART2_Init 0 */
    /* USER CODE END USART2_Init 0 */
    /* USER CODE BEGIN USART2_Init 1 */
    /* USER CODE END USART2_Init 1 */
    huart2.Instance = USART2;
    huart2.Init.BaudRate = 115200;
    huart2.Init.WordLength = UART_WORDLENGTH_8B;
    huart2.Init.StopBits = UART_STOPBITS_1;
    huart2.Init.Parity = UART_PARITY_NONE;
    huart2.Init.Mode = UART_MODE_TX_RX;
    huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart2.Init.OverSampling = UART_OVERSAMPLING_16;

```

```

if (HAL_UART_Init(&huart2) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN USART2_Init 2 */
/* USER CODE END USART2_Init 2 */
}
/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */
    /* USER CODE END MX_GPIO_Init_1 */
    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();
    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
    /*Configure GPIO pin : B1_Pin */
    GPIO_InitStruct.Pin = B1_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);
    /*Configure GPIO pin : LD2_Pin */
    GPIO_InitStruct.Pin = LD2_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);
    /* USER CODE BEGIN MX_GPIO_Init_2 */
    /* USER CODE END MX_GPIO_Init_2 */
}
/* USER CODE BEGIN 4 */
/* USER CODE END 4 */
/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();

```

```

while (1)
{
}
/* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 *        where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
/* USER CODE BEGIN 6 */
/* User can add his own implementation to report the file name and line number,
ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
/* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```

Receiver





Receiver\_main.c

```

/* USER CODE BEGIN Header */
/**
 * *****
 * @file      : main.c
 * @brief     : Main program body
 * *****
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 * *****
 */
/* USER CODE END Header */
/* Includes ----- */
#include "main.h"
#include <stdio.h>
#include <string.h>
/* Private includes ----- */
/* USER CODE BEGIN Includes */
/* USER CODE END Includes */
/* Private typedef ----- */
/* USER CODE BEGIN PTD */
/* USER CODE END PTD */
/* Private define ----- */
/* USER CODE BEGIN PD */
/* USER CODE END PD */
/* Private macro ----- */
/* USER CODE BEGIN PM */
/* USER CODE END PM */
/* Private variables ----- */
CAN_HandleTypeDef hcan1;
I2C_HandleTypeDef hi2c1;
UART_HandleTypeDef huart2;
/* USER CODE BEGIN PV */
CAN_RxHeaderTypeDef RxHeader;
CAN_FilterTypeDef canfilter;
uint8_t RxData[8];
float Ax,Ay,Az;
float Gx,Gy,Gz;
float DS_Temp;
int Speed;
uint8_t Temp;
uint8_t Hum;
uint8_t MPU_Fault = 1;

```

```

uint8_t DHT_Fault = 1;
uint8_t DS_Fault = 1;
uint8_t Speed_Fault = 1;
uint32_t MPU_LastRx = 0;
uint32_t DHT_LastRx = 0;
uint32_t DS_LastRx = 0;
uint32_t Speed_LastRx = 0;
char msg[300];
/* USER CODE END PV */
/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_CAN1_Init(void);
static void MX_I2C1_Init(void);
/* USER CODE BEGIN PFP */
void CAN_Filter_Config(void);
/* USER CODE END PFP */
/* Private user code -----*/
/* USER CODE BEGIN 0 */
/* USER CODE END 0 */
/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */
    /* USER CODE END 1 */
    /* MCU Configuration-----*/
    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();
    /* USER CODE BEGIN Init */
    /* USER CODE END Init */
    /* Configure the system clock */
    SystemClock_Config();
    /* USER CODE BEGIN SysInit */
    /* USER CODE END SysInit */
    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    MX_CAN1_Init();
    MX_I2C1_Init();
    /* USER CODE BEGIN 2 */
    CAN_Filter_Config();
    if(HAL_CAN_Start(&hcan1) != HAL_OK)
    {
        Error_Handler();
    }
}

```

```

HAL_CAN_ActivateNotification(&hcan1,
                             CAN_IT_RX_FIFO0_MSG_PENDING);
sprintf(msg,
        "CAN Receiver Ready\r\n");
HAL_UART_Transmit(&huart2,
                 (uint8_t *)msg,
                 strlen(msg),
                 HAL_MAX_DELAY);
/* USER CODE END 2 */
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */
    uint32_t now = HAL_GetTick();
    MPU_Fault = ((now - MPU_LastRx) > 1000);
    DHT_Fault = ((now - DHT_LastRx) > 1000);
    DS_Fault = ((now - DS_LastRx) > 1000);
    Speed_Fault = ((now - Speed_LastRx) > 1000);
    snprintf(msg, sizeof(msg),
            "AX=%.2f AY=%.2f AZ=%.2f\r\n"
            "GX=%.2f GY=%.2f GZ=%.2f\r\n"
            "MPU6050=%s\r\n"
            "TEMP=%d C HUM=%d %% DHT=%s\r\n"
            "DS18B20=%.2f C DS=%s\r\n"
            "Speed=%d km/h SpeedECU=%s\r\n\r\n",
            Ax, Ay, Az,
            Gx, Gy, Gz,
            MPU_Fault ? "FAULT" : "OK",
            Temp, Hum,
            DHT_Fault ? "FAULT" : "OK",
            DS_Temp,
            DS_Fault ? "FAULT" : "OK",
            Speed,
            Speed_Fault ? "FAULT" : "OK");
    HAL_UART_Transmit(&huart2,
                     (uint8_t *)msg,
                     strlen(msg),
                     HAL_MAX_DELAY);
    HAL_Delay(500);
    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}
/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)

```

```

{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    /** Configure the main internal regulator output voltage
    */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);
    /** Initializes the RCC Oscillators according to the specified parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    RCC_OscInitStruct.PLL.PLLM = 16;
    RCC_OscInitStruct.PLL.PLLN = 336;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV4;
    RCC_OscInitStruct.PLL.PLLQ = 2;
    RCC_OscInitStruct.PLL.PLLR = 2;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
    /** Initializes the CPU, AHB and APB buses clocks
    */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLOCK
        |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
    {
        Error_Handler();
    }
}
/**
 * @brief CAN1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_CAN1_Init(void)
{
    /* USER CODE BEGIN CAN1_Init 0 */
    /* USER CODE END CAN1_Init 0 */
    /* USER CODE BEGIN CAN1_Init 1 */
    /* USER CODE END CAN1_Init 1 */
    hcan1.Instance = CAN1;

```

```

hcan1.Init.Prescaler = 6;
hcan1.Init.Mode = CAN_MODE_NORMAL;
hcan1.Init.SyncJumpWidth = CAN_SJW_1TQ;
hcan1.Init.TimeSeg1 = CAN_BS1_11TQ;
hcan1.Init.TimeSeg2 = CAN_BS2_2TQ;
hcan1.Init.TimeTriggeredMode = DISABLE;
hcan1.Init.AutoBusOff = ENABLE;
hcan1.Init.AutoWakeUp = DISABLE;
hcan1.Init.AutoRetransmission = ENABLE;
hcan1.Init.ReceiveFifoLocked = DISABLE;
hcan1.Init.TransmitFifoPriority = DISABLE;
if (HAL_CAN_Init(&hcan1) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN CAN1_Init 2 */
/* USER CODE END CAN1_Init 2 */
}
/**
 * @brief I2C1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_I2C1_Init(void)
{
    /* USER CODE BEGIN I2C1_Init 0 */
    /* USER CODE END I2C1_Init 0 */
    /* USER CODE BEGIN I2C1_Init 1 */
    /* USER CODE END I2C1_Init 1 */
    hi2c1.Instance = I2C1;
    hi2c1.Init.ClockSpeed = 100000;
    hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
    hi2c1.Init.OwnAddress1 = 0;
    hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
    hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
    hi2c1.Init.OwnAddress2 = 0;
    hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
    hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
    if (HAL_I2C_Init(&hi2c1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN I2C1_Init 2 */
    /* USER CODE END I2C1_Init 2 */
}
/**
 * @brief USART2 Initialization Function
 * @param None
 * @retval None

```

```

*/
static void MX_USART2_UART_Init(void)
{
    /* USER CODE BEGIN USART2_Init 0 */
    /* USER CODE END USART2_Init 0 */
    /* USER CODE BEGIN USART2_Init 1 */
    /* USER CODE END USART2_Init 1 */
    huart2.Instance = USART2;
    huart2.Init.BaudRate = 115200;
    huart2.Init.WordLength = UART_WORDLENGTH_8B;
    huart2.Init.StopBits = UART_STOPBITS_1;
    huart2.Init.Parity = UART_PARITY_NONE;
    huart2.Init.Mode = UART_MODE_TX_RX;
    huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart2.Init.OverSampling = UART_OVERSAMPLING_16;
    if (HAL_UART_Init(&huart2) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN USART2_Init 2 */
    /* USER CODE END USART2_Init 2 */
}
/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */
    /* USER CODE END MX_GPIO_Init_1 */
    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();
    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
    /*Configure GPIO pin : B1_Pin */
    GPIO_InitStruct.Pin = B1_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);
    /*Configure GPIO pin : LD2_Pin */
    GPIO_InitStruct.Pin = LD2_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;

```



```

HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);
/* USER CODE BEGIN MX_GPIO_Init_2 */
/* USER CODE END MX_GPIO_Init_2 */
}
/* USER CODE BEGIN 4 */
void CAN_Filter_Config(void)
{
    canfilter.FilterActivation = ENABLE;
    canfilter.FilterBank = 0;
    canfilter.FilterFIFOAssignment =
        CAN_FILTER_FIFO0;
    canfilter.FilterIdHigh = 0;
    canfilter.FilterIdLow = 0;
    canfilter.FilterMaskIdHigh = 0;
    canfilter.FilterMaskIdLow = 0;
    canfilter.FilterMode =
        CAN_FILTERMODE_IDMASK;
    canfilter.FilterScale =
        CAN_FILTERSCALE_32BIT;
    canfilter.SlaveStartFilterBank = 14;
    if(HAL_CAN_ConfigFilter(&hcan1,
        &canfilter) != HAL_OK)
    {
        Error_Handler();
    }
}
void HAL_CAN_RxFifo0MsgPendingCallback(
    CAN_HandleTypeDef *hcan)
{
    int16_t temp1, temp2, temp3;
    HAL_CAN_GetRxMessage(hcan,
        CAN_RX_FIFO0,
        &RxHeader,
        RxData);
    switch(RxHeader.StdId)
    {
        /* MPU6050 Accelerometer */
        case 0x100:
            temp1 = (int16_t)((RxData[0] << 8) | RxData[1]);
            temp2 = (int16_t)((RxData[2] << 8) | RxData[3]);
            temp3 = (int16_t)((RxData[4] << 8) | RxData[5]);
            Ax = (float)temp1 / 100.0f;
            Ay = (float)temp2 / 100.0f;
            Az = (float)temp3 / 100.0f;
            MPU_LastRx = HAL_GetTick();
            break;
        /* MPU6050 Gyroscope */
        case 0x101:

```

```

temp1 = (int16_t)((RxData[0] << 8) | RxData[1]);
temp2 = (int16_t)((RxData[2] << 8) | RxData[3]);
temp3 = (int16_t)((RxData[4] << 8) | RxData[5]);
Gx = (float)temp1 / 100.0f;
Gy = (float)temp2 / 100.0f;
Gz = (float)temp3 / 100.0f;
MPU_LastRx = HAL_GetTick();
break;
/* DHT11 */
case 0x201:
    Temp = RxData[0];
    Hum = RxData[1];
    DHT_LastRx = HAL_GetTick();
    break;
case 0x301:
    temp1 = (int16_t)((RxData[0] << 8)
                    | RxData[1]);
    DS_Temp = (float)temp1 / 100.0f;
    DS_LastRx = HAL_GetTick();
    break;
case 0x401:
    Speed = (int16_t)((RxData[0] << 8)
                    | RxData[1]);
    Speed_LastRx = HAL_GetTick();
/*
    char MPU_Status[10];
    char DHT_Status[10];
    char DS_Status[10];
    char Speed_Status[10];
    strcpy(MPU_Status, (MPU_Fault == 0) ? "OK" : "FAULT");
    strcpy(DHT_Status, (DHT_Fault == 0) ? "OK" : "FAULT");
    strcpy(DS_Status, (DS_Fault == 0) ? "OK" : "FAULT");
    strcpy(Speed_Status, (Speed_Fault == 0) ? "OK" : "FAULT");
    snprintf(msg, sizeof(msg),
             "AX=%.2f AY=%.2f AZ=%.2f\r\n"
             "GX=%.2f GY=%.2f GZ=%.2f\r\n"
             "MPU6050=%s\r\n"
             "TEMP=%d C HUM=%d %% DHT=%s\r\n"
             "DS18B20=%.2f C DS=%s\r\n"
             "Speed=%d SpeedECU=%s\r\n\r\n",
             Ax, Ay, Az,
             Gx, Gy, Gz,
             MPU_Status,
             Temp, Hum, DHT_Status,
             DS_Temp, DS_Status,
             Speed, Speed_Status);
    HAL_UART_Transmit(&huart2,
                     (uint8_t*)msg,
                     strlen(msg),
                     HAL_MAX_DELAY);*/

```

```

        break;
/* Unknown CAN ID */
default:
    sprintf(msg,
            "Unknown ID: 0x%03lX\r\n",
            RxHeader.StdId);
    HAL_UART_Transmit(&huart2,
                      (uint8_t *)msg,
                      strlen(msg),
                      HAL_MAX_DELAY);

    break;
}
}
/* USER CODE END 4 */
/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```

# Ethernet Gateway



Transmitter



config.py



```
# --- SPI / MCP2515 ---
SPI_BUS = 0
SPI_DEVICE = 0          # 0 = CE0 (GPIO8), 1 = CE1 (GPIO7)
SPI_SPEED_HZ = 1_000_000
OSC_MHZ = 8
CAN_BITRATE = 500000    # must match the STM32's bxCAN configuration

# --- UDP ---
LAPTOP_IP = "192.168.1.100"  # <-- change to your laptop's actual IP
LAPTOP_PORT = 5000
```

mcp2515\_regs.py

"""

MCP2515 register map and SPI instruction set.

Reference: Microchip MCP2515 datasheet (DS20001801).

"""

# ---- SPI Instructions ----

```
INSTR_RESET      = 0xC0
INSTR_READ       = 0x03
INSTR_WRITE      = 0x02
INSTR_RTS        = 0x80    # OR with 0x01/0x02/0x04 for TXB0/1/2
INSTR_READ_STATUS = 0xA0
INSTR_RX_STATUS  = 0xB0
INSTR_BIT_MODIFY  = 0x05
INSTR_READ_RXB0   = 0x90    # read RX buffer 0, starting at SIDH
INSTR_READ_RXB1   = 0x94    # read RX buffer 1, starting at SIDH
INSTR_LOAD_TXB0   = 0x40    # load TX buffer 0, starting at SIDH
```

# ---- Control / Status Registers ----

```
CANSTAT  = 0x0E
CANCTRL  = 0x0F
CNF3     = 0x28
CNF2     = 0x29
CNF1     = 0x2A
CANINTE  = 0x2B
CANINTF  = 0x2C
EFLG     = 0x2D
TEC      = 0x1C
REC      = 0x1D
```

# ---- RX Buffer 0 ----

```
RXB0CTRL = 0x60
RXB0SIDH = 0x61
RXB0SIDL = 0x62
RXB0EID8 = 0x63
RXB0EID0 = 0x64
RXB0DLC  = 0x65
RXB0D0   = 0x66
```

# ---- RX Buffer 1 ----

```
RXB1CTRL = 0x70
RXB1SIDH = 0x71
RXB1SIDL = 0x72
RXB1EID8 = 0x73
RXB1EID0 = 0x74
RXB1DLC  = 0x75
RXB1D0   = 0x76
```

```

# ---- TX Buffer 0 ----
TXB0CTRL  = 0x30
TXB0SIDH  = 0x31
TXB0SIDL  = 0x32
TXB0EID8  = 0x33
TXB0EID0  = 0x34
TXB0DLC   = 0x35
TXB0D0    = 0x36

# ---- Masks / Filters (only need RXM0 for "accept everything") ----
RXM0SIDH  = 0x20
RXM0SIDL  = 0x21
RXM1SIDH  = 0x24
RXM1SIDL  = 0x25

# ---- CANCTRL: REQOP bits [7:5] ----
MODE_NORMAL      = 0x00
MODE_SLEEP       = 0x20
MODE_LOOPBACK    = 0x40
MODE_LISTENONLY  = 0x60
MODE_CONFIG      = 0x80
MODE_MASK        = 0xE0

# ---- CANSTAT: OPMOD bits [7:5], read-only mirror of current mode ----
OPMOD_MASK = 0xE0

# ---- CANINTF flag bits ----
RX0IF = 0x01
RX1IF = 0x02
TX0IF = 0x04
TX1IF = 0x08
TX2IF = 0x10
ERRIF = 0x20
WAKIF = 0x40
MERRF = 0x80

# ---- RXB0CTRL ----
# RXM<1:0> = 11 -> receive any message, filters/masks disabled
RXB0CTRL_RXM_ANY = 0x60
RXB0CTRL_BUKT    = 0x04    # rollover to RXB1 if RXB0 full

# ---- RXB1CTRL ----
RXB1CTRL_RXM_ANY = 0x60

# ---- Bit timing configuration tables ----
# (CNF1, CNF2, CNF3) for an 8 MHz crystal, common bitrates.
# Values taken from the standard MCP2515 timing tables used across

```

```
# widely-deployed open-source drivers (e.g. autowp/mcp2515, Seeed-Studio).
BITRATE_8MHZ = {
    125000: (0x01, 0xB1, 0x85),
    250000: (0x00, 0xB1, 0x85),
    500000: (0x00, 0x90, 0x02),
    1000000: (0x00, 0x80, 0x00),
}
```

mcp2515.py

```

"""
Direct-SPI MCP2515 driver for Raspberry Pi (bypasses SocketCAN entirely).

Requires: pip3 install spidev

Wiring reminder (Pi 5, SPI0):
MCP2515 VCC -> 5V (module has onboard regulator to 3.3V logic)
MCP2515 GND -> GND
MCP2515 SCK -> GPIO11 (SPI0 SCLK)
MCP2515 SI  -> GPIO10 (SPI0 MOSI)
MCP2515 SO  -> GPIO9  (SPI0 MISO)
MCP2515 CS  -> GPIO8  (SPI0 CE0)   [or GPIO7/CE1, set bus/device below]
MCP2515 INT -> any free GPIO (optional, for interrupt-driven RX)

"""

import time
import spidev

from mcp2515_regs import *

class CANFrame:
    """Simple container for a received/sent CAN frame."""

    def __init__(self, arb_id, data, dlc=None, extended=False):
        self.arb_id = arb_id
        self.data = bytes(data)
        self.dlc = dlc if dlc is not None else len(data)
        self.extended = extended

    def __repr__(self):
        hexdata = " ".join(f"{b:02X}" for b in self.data)
        idfmt = f"{self.arb_id:08X}" if self.extended else
f"{self.arb_id:03X}"
        return f"CANFrame(ID=0x{idfmt} DLC={self.dlc} DATA=[{hexdata}])"

class MCP2515:
    def __init__(self, bus=0, device=0, speed_hz=1_000_000, osc_mhz=8,
        bitrate=500000):
        """
        bus/device: SPI bus/CE line (bus=0, device=0 -> /dev/spidev0.0)
        speed_hz:   SPI clock speed. Keep <= 10 MHz; 1 MHz is a safe
start.
        osc_mhz:    Crystal/oscillator frequency on your MCP2515 module.
        bitrate:    Desired CAN bus bitrate (must match STM32 side).

```

```

"""
if osc_mhz != 8:
    raise ValueError(
        "Only the 8 MHz timing table is included. Add your own "
        "CNF1/CNF2/CNF3 values to BITRATE_8MHZ-style table in "
        "mcp2515_regs.py if your module uses a different crystal."
    )
if bitrate not in BITRATE_8MHZ:
    raise ValueError(f"No timing entry for {bitrate} bps at 8
MHz")

self.cnf1, self.cnf2, self.cnf3 = BITRATE_8MHZ[bitrate]

self.spi = spidev.SpiDev()
self.spi.open(bus, device)
self.spi.max_speed_hz = speed_hz
self.spi.mode = 0b00 # MCP2515 requires SPI mode 0

# ----- Low-level SPI helpers -----

def reset(self):
    """Issue the SPI RESET instruction; puts the chip in config
mode."""
    self.spi.xfer2([INSTR_RESET])
    time.sleep(0.01) # datasheet: allow >2us, we give it more margin

def read_register(self, addr):
    resp = self.spi.xfer2([INSTR_READ, addr, 0x00])
    return resp[2]

def read_registers(self, addr, n):
    resp = self.spi.xfer2([INSTR_READ, addr] + [0x00] * n)
    return resp[2:]

def write_register(self, addr, value):
    self.spi.xfer2([INSTR_WRITE, addr, value])

def write_registers(self, addr, values):
    self.spi.xfer2([INSTR_WRITE, addr] + list(values))

def bit_modify(self, addr, mask, value):
    """Only works on a subset of registers per datasheet (CANCTRL,
CANINTF, RXBnCTRL, etc). Not valid on every register."""
    self.spi.xfer2([INSTR_BIT_MODIFY, addr, mask, value])

def read_status(self):
    return self.spi.xfer2([INSTR_READ_STATUS, 0x00])[1]

```



```

def request_to_send(self, txb=0):
    self.spi.xfer2([INSTR_RTS | (1 << txb)])

# ----- Mode control -----

def set_mode(self, mode):
    self.bit_modify(CANCTRL, MODE_MASK, mode)
    # Poll CANSTAT until the chip confirms the mode switch
    deadline = time.time() + 0.5
    while time.time() < deadline:
        if (self.read_register(CANSTAT) & OPMOD_MASK) == mode:
            return
        time.sleep(0.001)
    raise TimeoutError("MCP2515 did not switch mode in time")

# ----- Initialization -----

def begin(self):
    """Full chip init: reset, configure bit timing, open RX filters,
    drop into normal mode."""
    self.reset()
    self.set_mode(MODE_CONFIG)

    # Bit timing
    self.write_register(CNF1, self.cnf1)
    self.write_register(CNF2, self.cnf2)
    self.write_register(CNF3, self.cnf3)

    # Disable interrupts (we'll poll CANINTF instead of using the
    # hardware INT pin, to keep this example self-contained)
    self.write_register(CANINTE, 0x00)

    # Accept every message on both receive buffers, no filtering.
    # RXM<1:0> = 11 disables mask/filter checking entirely.
    self.write_register(RXB0CTRL, RXB0CTRL_RXM_ANY | RXB0CTRL_BUKT)
    self.write_register(RXB1CTRL, RXB1CTRL_RXM_ANY)

    # Clear any stale interrupt flags
    self.write_register(CANINTF, 0x00)

    self.set_mode(MODE_NORMAL)

# ----- Receiving -----

def _read_rx_buffer(self, buffer_num):
    """Read a full frame out of RXB0 or RXB1."""

```

```

instr = INSTR_READ_RXB0 if buffer_num == 0 else INSTR_READ_RXB1
# SIDH, SIDL, EID8, EID0, DLC, D0..D7 (13 bytes after the opcode)
resp = self.spi.xfer2([instr] + [0x00] * 13)
sidh, sidl, eid8, eid0, dlc = resp[1:6]
data = resp[6:6 + (dlc & 0x0F)]

extended = bool(sidl & 0x08)
if extended:
    arb_id = (sidh << 21) | ((sidl & 0xE0) << 13) | \
            ((sidl & 0x03) << 16) | (eid8 << 8) | eid0
else:
    arb_id = (sidh << 3) | (sidl >> 5)

return CANFrame(arb_id, data, dlc=dlc & 0x0F, extended=extended)

def read_pending_frame(self):
    """Return one CANFrame if available, else None (non-blocking)."""
    status = self.read_status()
    # READ STATUS bit layout: bit0=RX0IF, bit1=RX1IF (see datasheet
    # Table 12-4 for the rest, not needed here)
    if status & 0x01:
        frame = self._read_rx_buffer(0)
        self.bit_modify(CANINTF, RX0IF, 0x00)
        return frame
    if status & 0x02:
        frame = self._read_rx_buffer(1)
        self.bit_modify(CANINTF, RX1IF, 0x00)
        return frame
    return None

def recv(self, poll_interval=0.001, timeout=None):
    """Blocking receive: polls until a frame arrives or timeout (s)
    elapses. Returns a CANFrame, or None on timeout."""
    deadline = None if timeout is None else time.time() + timeout
    while True:
        frame = self.read_pending_frame()
        if frame is not None:
            return frame
        if deadline is not None and time.time() > deadline:
            return None
        time.sleep(poll_interval)

# ----- Sending (bonus, not required for your gateway) -----

def send(self, arb_id, data, extended=False):
    data = list(data)[:8]
    dlc = len(data)

```

```

if extended:
    sidh = (arb_id >> 21) & 0xFF
    sidl = (((arb_id >> 18) & 0x07) << 5) | 0x08 | \
            ((arb_id >> 16) & 0x03)
    eid8 = (arb_id >> 8) & 0xFF
    eid0 = arb_id & 0xFF
else:
    sidh = (arb_id >> 3) & 0xFF
    sidl = (arb_id & 0x07) << 5
    eid8 = 0x00
    eid0 = 0x00

payload = [sidh, sidl, eid8, eid0, dlc] + data
self.spi.xfer2([INSTR_LOAD_TXB0] + payload)
self.request_to_send(txb=0)

def close(self):
    self.spi.close()

```

can\_udp\_gateway.py

```

"""
CAN -> Ethernet gateway, SocketCAN-bypass version.

Reads CAN frames directly off the MCP2515 over SPI (no can0 interface
involved at all) and forwards each frame to your laptop over UDP.

Run on the Raspberry Pi:
    python3 can_udp_gateway.py
"""

import socket
import sys

from mcp2515 import MCP2515
import config

def main():
    print("Initializing MCP2515 (direct SPI, no SocketCAN)...")
    can = MCP2515(
        bus=config.SPI_BUS,
        device=config.SPI_DEVICE,
        speed_hz=config.SPI_SPEED_HZ,
        osc_mhz=config.OSC_MHZ,
        bitrate=config.CAN_BITRATE,
    )

    try:
        can.begin()
    except TimeoutError:
        print("ERROR: MCP2515 did not respond. Check wiring, CS line, "
              "and that no kernel mcp251x driver is holding the SPI bus.")
        sys.exit(1)

    print(f"MCP2515 up at {config.CAN_BITRATE} bps. Listening...")

    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    try:
        while True:
            frame = can.recv(timeout=None)
            if frame is None:
                continue

            msg = (
                f"ID={hex(frame.arb_id)} "
                f"DLC={frame.dlc} "

```

```
        f"DATA={' '.join(f'{b:02X}' for b in frame.data)}"
    )
    sock.sendto(msg.encode(), (config.LAPTOP_IP,
config.LAPTOP_PORT))
    print("Sent:", msg)

except KeyboardInterrupt:
    print("\nStopping.")
finally:
    sock.close()
    can.close()

if __name__ == "__main__":
    main()
```

Receiver.py

```

import socket
import threading
import tkinter as tk
from tkinter import ttk

# -----
# UDP Configuration
# -----
HOST = "0.0.0.0"
PORT = 5000

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((HOST, PORT))

# -----
# GUI
# -----
root = tk.Tk()
root.title("CAN Sensor Dashboard")
root.geometry("650x500")
root.resizable(False, False)

title = tk.Label(
    root,
    text="CAN SENSOR DASHBOARD",
    font=("Arial", 18, "bold"),
    fg="blue"
)
title.pack(pady=10)

frame = ttk.Frame(root, padding=20)
frame.pack(fill="both", expand=True)

# -----
# Variables
# -----
accX = tk.StringVar(value="0.00 g")
accY = tk.StringVar(value="0.00 g")
accZ = tk.StringVar(value="0.00 g")

gyroX = tk.StringVar(value="0.00")
gyroY = tk.StringVar(value="0.00")
gyroZ = tk.StringVar(value="0.00")

```



```

dhtTemp = tk.StringVar(value="0 °C")
humidity = tk.StringVar(value="0 %")

dsTemp = tk.StringVar(value="0.00 °C")

speed = tk.StringVar(value="0 km/h")

status = tk.StringVar(value="Waiting...")

# -----
# Frames
# -----
accFrame = ttk.LabelFrame(frame, text="Accelerometer")
accFrame.grid(row=0, column=0, padx=10, pady=10)

gyroFrame = ttk.LabelFrame(frame, text="Gyroscope")
gyroFrame.grid(row=0, column=1, padx=10, pady=10)

dhtFrame = ttk.LabelFrame(frame, text="DHT11")
dhtFrame.grid(row=1, column=0, padx=10, pady=10)

dsFrame = ttk.LabelFrame(frame, text="DS18B20")
dsFrame.grid(row=1, column=1, padx=10, pady=10)

speedFrame = ttk.LabelFrame(frame, text="Speed")
speedFrame.grid(row=2, column=0, columnspan=2, padx=10, pady=10, sticky="ew")

def add_row(parent, name, var, row):
    ttk.Label(parent, text=name, width=12).grid(row=row, column=0, sticky="w")
    ttk.Label(parent,
               textvariable=var,
               font=("Arial", 11, "bold"),
               foreground="blue").grid(row=row, column=1, sticky="w")

# Accelerometer
add_row(accFrame, "X", accX, 0)
add_row(accFrame, "Y", accY, 1)
add_row(accFrame, "Z", accZ, 2)

# Gyroscope
add_row(gyroFrame, "X", gyroX, 0)
add_row(gyroFrame, "Y", gyroY, 1)

```

```

add_row(gyroFrame, "Z", gyroZ, 2)

# DHT11
add_row(dhtFrame, "Temp", dhtTemp, 0)
add_row(dhtFrame, "Humidity", humidity, 1)

# DS18B20
add_row(dsFrame, "Temp", dsTemp, 0)

# Speed
add_row(speedFrame, "Speed", speed, 0)

statusLabel = ttk.Label(root,
                        textvariable=status,
                        font=("Arial", 11))
statusLabel.pack()

# -----
# Receiver Thread
# -----
def receiver():

    while True:

        packet, addr = sock.recvfrom(1024)

        message = packet.decode().strip()

        print(message)

    try:

        # Example:
        # ID=0x100 DLC=6 DATA=00 05 ff fe 00 6a

        parts = message.split()

        # CAN ID
        can_id = int(parts[0].split("=")[1], 16)

        # DLC
        dlc = int(parts[1].split("=")[1])

```

```

# DATA bytes
data_list = []

# First byte after DATA=
data_list.append(parts[2].split("=")[1])

# Remaining bytes
for i in range(3, len(parts)):
    data_list.append(parts[i])

# Convert to bytes
data = bytes(int(x, 16) for x in data_list)

# Verify length
if len(data) != dlc:
    print("Length mismatch")
    continue

# -----
# Accelerometer
# -----
if can_id == 0x100:

    ax = int.from_bytes(data[0:2], 'big', signed=True) / 100
    ay = int.from_bytes(data[2:4], 'big', signed=True) / 100
    az = int.from_bytes(data[4:6], 'big', signed=True) / 100

    accX.set(f"{ax:.2f} g")
    accY.set(f"{ay:.2f} g")
    accZ.set(f"{az:.2f} g")

# -----
# Gyroscope
# -----
elif can_id == 0x101:

    gx = int.from_bytes(data[0:2], 'big', signed=True) / 100
    gy = int.from_bytes(data[2:4], 'big', signed=True) / 100
    gz = int.from_bytes(data[4:6], 'big', signed=True) / 100

    gyroX.set(f"{gx:.2f}")
    gyroY.set(f"{gy:.2f}")
    gyroZ.set(f"{gz:.2f}")

```

```

# -----
# DHT11
# -----
elif can_id == 0x201:

    dhtTemp.set(f"{data[0]} °C")
    humidity.set(f"{data[1]} %")

# -----
# DS18B20
# -----
elif can_id == 0x301:

    temp = int.from_bytes(data[0:2], 'big', signed=True) / 100

    dsTemp.set(f"{temp:.2f} °C")

# -----
# Speed
# -----
elif can_id == 0x401:

    spd = int.from_bytes(data[0:2], 'big')

    speed.set(f"{spd} km/h")

status.set("Receiving CAN Frames")

except Exception as e:
    print("ERROR:", e)

threading.Thread(target=receiver, daemon=True).start()

root.mainloop()

```